

Network and Cloud Infrastructures Project Work - SDN

Network Slicing with Ryu and Mininet

Davide Di Matteo - M63001865
dav.dimatteo@studenti.unina.it

Università degli studi di Napoli Federico II — 28/03/2026

Introduzione

The project implements network slicing in a virtualised SDN environment deployed on an **Ubuntu 22.04 LTS virtual machine**. The experimental setup is based on:

- **Mininet**, used to emulate hosts, switches, and links;
- **Open vSwitch** (via Mininet), used as the software data plane;
- **Python 3** scripts, used to define topologies and automate tests;
- **Ryu controller**, used to implement SDN control logic and OpenFlow rules.

The implementation is organised into the following sections:

1. **Topology-slicing**: restriction of communication paths between specific hosts;
2. **Service-slicing**: differentiated treatment of selected traffic classes.

For each section, validation tests are performed (e.g., connectivity and traffic checks) to evaluate the effectiveness of the proposed slicing policies.

1 Topology slicing

This section covers the definition of the network topology in Mininet and the resulting division into two isolated *slices*. The aim is to construct a topology in which hosts can communicate via predefined routes:

- The host h1 can communicate only with h3 via the **upper path**;
- The host h2 can communicate only with h4 via the **lower path**.

For this purpose, the following Mininet configuration has been defined by implementing the constructor of the `Mininet.Environment` class, whose logical view is represented in figure 1.

```
1 class Environment(object):
2     def __init__(self):
3         # ...
4         # Nodes definition
5         # Hosts definition
```

```

6 self.h1 = self.net.addHost('h1', mac= '00:00:00:00:00:01', ip= '10.0.0.1')
7 self.h2 = self.net.addHost('h2', mac= '00:00:00:00:00:02', ip= '10.0.0.2')
8 self.h3 = self.net.addHost('h3', mac= '00:00:00:00:00:03', ip= '10.0.0.3')
9 self.h4 = self.net.addHost('h4', mac= '00:00:00:00:00:04', ip= '10.0.0.4')
10 # Switches definition
11 self.s1 = self.net.addSwitch('s1')
12 self.s2 = self.net.addSwitch('s2')
13 self.Up = self.net.addSwitch('s3') # up
14 self.Dw = self.net.addSwitch('s4') # down
15
16 # Links definition
17 # Hosts to Switches
18 self.net.addLink(self.h1, self.s1, delay='0.01ms', port1=1, port2=1)
19 self.net.addLink(self.h2, self.s1, delay='0.01ms', port1=1, port2=2)
20 self.net.addLink(self.h3, self.s2, delay='0.01ms', port1=1, port2=3)
21 self.net.addLink(self.h4, self.s2, delay='0.01ms', port1=1, port2=4)
22 # Switches to Switches
23 self.net.addLink(self.s1, self.Up, bw=10, delay='0.025ms', port1=3, port2=1)
24 self.net.addLink(self.Up, self.s2, bw=10, delay='0.025ms', port1=2, port2=1)
25 self.net.addLink(self.s1, self.Dw, bw=1, delay='0.025ms', port1=4, port2=1)
26 self.net.addLink(self.Dw, self.s2, bw=1, delay='0.025ms', port1=2, port2=2)
27 # ...

```

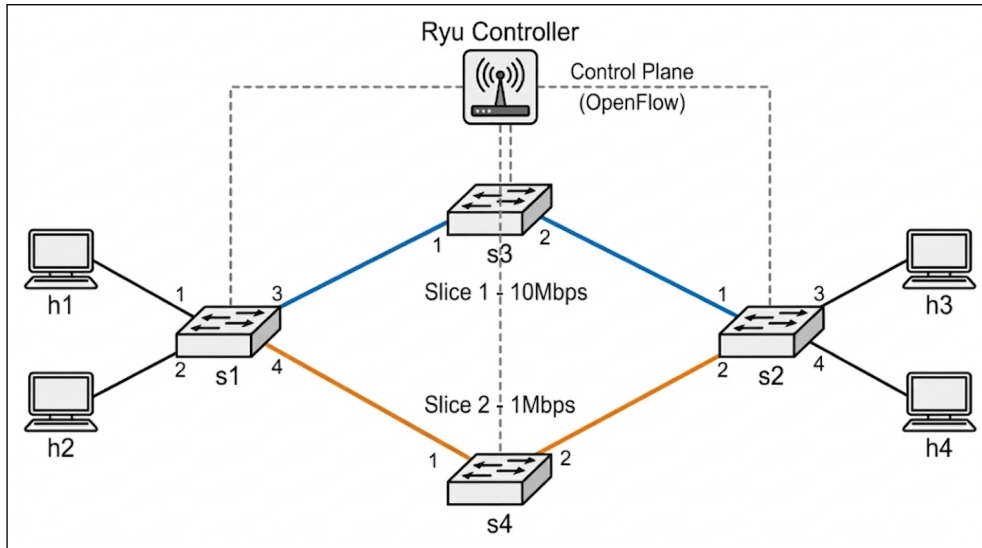


Figure 1: Logical view of the network topology

Given the defined network topology, a **deterministic** routing approach was implemented within the controller. This design ensures granular control over traffic flows, allowing for the enforcement of static routes that guarantee slice isolation and adherence to predefined network policies.

RyuController is an SDN application based on the OpenFlow 1.3 protocol, designed to manage a network environment emulated via Mininet and Open vSwitch. The application was developed by extending the `ryu.base.app_manager.RyuApp` base class. To ensure deterministic forwarding and strict slice isolation, a static addressing scheme was adopted: host MAC addresses are defined as constant strings directly within the class constructor."

```

1 # Base class extension
2 class RyuController(app_manager.RyuApp):
3     # OpenFlow 1.3
4     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
5
6     def __init__(self, *args, **kwargs):
7         super(RyuController, self).__init__(*args, **kwargs)
8
9         # Static MAC addresses definition for deterministic routing
10        self.h1 = '00:00:00:00:00:01'
11        self.h2 = '00:00:00:00:00:02'
12        self.h3 = '00:00:00:00:00:03'
13        self.h4 = '00:00:00:00:00:04'

```

```

14
15 # Utility function to add flow rules to the switches
16 def add_flow(self, datapath, priority, match, actions, idle_timeout=0, flag=0):
17     ofproto = datapath.ofproto
18     parser = datapath.ofproto_parser
19
20     inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
21     mod = parser.OFPFlowMod(
22         datapath=datapath, priority=priority,
23         match=match, instructions=inst,
24         idle_timeout = idle_timeout, flags=flag
25     )
26
27     datapath.send_msg(mod) # Send the flow modification message to the switch

```

The routing strategy implemented in this SDN controller follows a **proactive** approach. Rather than reacting to packets arriving at the controller, flow rules are pre-installed on all switches during the controller initialization phase, specifically when the `switch_features_handler` is triggered upon OpenFlow handshake completion.

This proactive method offers several advantages:

- **Deterministic behavior:** All routing decisions are made statically before any traffic flows through the network, ensuring predictable and consistent packet forwarding;
- **Reduced latency:** Packets are forwarded according to pre-installed rules without requiring controller interaction, eliminating the overhead of reactive packet-in messages;
- **Improved performance:** The controller is not burdened with runtime packet processing, allowing it to manage multiple switches more efficiently;
- **Enhanced slice isolation:** Static rules guarantee that traffic is strictly confined within designated slices, preventing any cross-slice communication regardless of runtime conditions.

Unlike reactive approaches where the controller would process unknown flows dynamically, this proactive strategy ensures that every switch has complete knowledge of how to forward packets for all host pairs, thus achieving the isolation guarantees required by the network slicing architecture. When Mininet initiates a switch, it attempts to establish a TCP connection with the Ryu controller. If the connection is successful (i.e. the three-way handshake is completed), the OpenFlow handshake is then initiated. Subsequently, the switch sends a `FEATURES_REPLY` message, and the controller executes the `switch_features_handler` function. This function is responsible for installing the flow rules in the switch, which are defined based on the switch's unique identifier (`dpid`). The flow rules are designed to enforce the predefined routing paths and ensure that traffic is confined within the designated slices of the network.

```

1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     ofproto = datapath.ofproto
5     parser = datapath.ofproto_parser
6     dpid = datapath.id
7
8     if dpid == 1: # Switch S1
9         # UP flow configuration
10        port_out_Up = 3 # S1 out port (UP)
11        port_in_Up = 1 # S1 in port (UP)
12
13        # Flow IN
14        self.add_mac_flow(datapath, self.h1, self.h3, port_out_Up) # Eth H1 to H3 via s1:1
15        self.add_arp_flow(datapath, self.h1, port_out_Up) # ARP H1 to H3 via s1:1
16
17        # Flow Back
18        self.add_mac_flow(datapath, self.h3, self.h1, port_in_Up) # Eth H3 to H1 via s1:3
19        self.add_arp_flow(datapath, self.h3, port_in_Up) # ARP from H3 to H1 via s1:3
20
21        #####
22
23        # Down flow configuration
24        port_out_Dw = 4 # S1 out port (DW)
25        port_in_Dw = 2 # S1 in port (DW)

```

```

26
27     # Flow IN
28     self.add_mac_flow(datapath, self.h2, self.h4, port_out_Dw) # Eth H2 to H4 via s1:2
29     self.add_arp_flow(datapath, self.h2, port_out_Dw) # ARP H2 to H4 via s1:2
30
31     # Flow Back
32     self.add_mac_flow(datapath, self.h4, self.h2, port_in_Dw) # Eth H4 to H2 via s1:4
33     self.add_arp_flow(datapath, self.h4, port_in_Dw) # ARP H4 to H2 via s1:4
34
35     elif dpid == 2: # Switch S2
36         # UP flow configuration
37         port_out_Up = 1 # S2 out port (UP)
38         port_in_Up = 3 # S2 in port (UP)
39
40         # Flow IN
41         self.add_mac_flow(datapath, self.h3, self.h1, port_out_Up) # Eth H3 to H1 via s2:1
42         self.add_arp_flow(datapath, self.h3, port_out_Up) # ARP H3 to H1 via s2:1
43
44         # Flow Back
45         self.add_mac_flow(datapath, self.h1, self.h3, port_in_Up) # Eth H1 to H3 via s2:3
46         self.add_arp_flow(datapath, self.h1, port_in_Up) # ARP H1 to H3 via s2:3
47
48         #####
49
50         # Down flow configuration
51         port_out_Dw = 2 # S2 out port (DW)
52         port_in_Dw = 4 # S2 in port (DW)
53
54         # Flow IN
55         self.add_mac_flow(datapath, self.h4, self.h2, port_out_Dw) # Eth H4 to H2 via s2:2
56         self.add_arp_flow(datapath, self.h4, port_out_Dw) # ARP H4 to H2 via s2:2
57
58         # Flow Back
59         self.add_mac_flow(datapath, self.h2, self.h4, port_in_Dw) # Eth H2 to H4 via s2:4
60         self.add_arp_flow(datapath, self.h2, port_in_Dw) # ARP H2 to H4 via s2:4
61
62     elif dpid == 3: # Switch 3 (UP)
63         port_1 = 1 # S3 in port (UP)
64         port_2 = 2 # S3 out port (UP)
65
66         self.add_mac_flow(datapath, self.h1, self.h3, port_2) # Eth H1 to H3 via s3:2
67         self.add_arp_flow(datapath, self.h1, port_2) # ARP H1 to H3 via s3:2
68
69         self.add_mac_flow(datapath, self.h3, self.h1, port_1) # Eth H3 to H1 via s3:1
70         self.add_arp_flow(datapath, self.h3, port_1) # ARP H3 to H1 via s3:1
71
72
73     elif dpid == 4: # Switch 4 (DW)
74         port_1 = 1 # S4 in port (DW)
75         port_2 = 2 # S4 out port (DW)
76
77         self.add_mac_flow(datapath, self.h2, self.h4, port_2) # Eth H2 to H4 via s4:2
78         self.add_arp_flow(datapath, self.h2, port_2) # ARP H2 to H4 via s4:2
79
80         self.add_mac_flow(datapath, self.h4, self.h2, port_1) # Eth H4 to H2 via s4:1
81         self.add_arp_flow(datapath, self.h4, port_1) # ARP H4 to H2 via s4:1

```

After determining the switch identity with *dpid*, the controller creates the rule through `add_mac_flow`, which is the association $(\text{mac_src}, \text{mac_dst}) \rightarrow \text{port}$. Through the `add_arp_flow` rule, we enforce that ARP messages can exit on multiple ports, and it is a rule with higher priority than the one that enforces MAC-based flow. We observe that the ARP packet, which is sent in broadcast, is nevertheless confined within the network *slice*.

```

1     def add_mac_flow(self, datapath, src, dst, out_port, priority=10): # Utility function
2         # to add flow rules based on MAC addresses, specifying the source and destination
3         # MAC addresses, the output port, and an optional priority level for the flow rule
4         parser = datapath.ofproto_parser
5
6         match = parser.OFPMatch(eth_src=src, eth_dst=dst)
7         actions = [parser.OFPActionOutput(out_port)]
8         self.add_flow(datapath, priority, match, actions)

```

```

7
8 def add_arp_flow(self, datapath, eth_src, out_port): # Utility function to add flow
    rules for ARP packets, matching only the source MAC address and allowing the
    packet to be output on multiple ports (if out_port is a list), with a higher
    priority than the MAC-based flow rules to ensure that ARP packets are handled
    correctly
9 parser = datapath.ofproto_parser
10
11 match = parser.OFPMatch(eth_src=eth_src, eth_dst="ff:ff:ff:ff:ff:ff", eth_type=0
    x0806)
12 actions = []
13 if not isinstance(out_port, list):
14     out_port = [out_port]
15
16 for port in out_port:
17     actions.append(parser.OFPACTIONOutput(port))
18 self.add_flow(datapath, priority=20, match=match, actions=actions)

```

To test this configuration, Mininet's *pingall* tool and *Wireshark* were used.

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)

```

Figure 2: pingall results: hosts h1 and h3 can communicate, while hosts h2 and h4 can communicate, confirming the isolation of the two slices.

9	24.400751813	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.1
10	25.408132633	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.1
11	26.432216400	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.1
12	27.462619812	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.1
13	27.463452437	00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 10.0.0.3 is at 00:00:00:00:00:03
14	27.464052836	10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request id=0x1d24, seq=1/256, ttl=64 (reply in 15)
15	27.464784378	10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply id=0x1d24, seq=1/256, ttl=64 (request in 14)
16	27.469362934	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.1
17	28.481793976	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.1
18	29.505254716	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.1
19	32.770710369	00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 Who has 10.0.0.1? Tell 10.0.0.3
20	32.773166985	00:00:00_00:00:01	00:00:00_00:00:03	ARP	42 10.0.0.1 is at 00:00:00:00:00:01
21	36.750005723	10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) request id=0x010a, seq=1/256, ttl=64 (reply in 22)
22	36.750309553	10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) reply id=0x010a, seq=1/256, ttl=64 (request in 21)
23	36.755894448	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.3
24	37.761834881	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.3
25	38.784793016	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.3
26	39.812623014	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.3
27	40.832990270	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.3
28	41.857043208	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.3

Figure 3: Wireshark capture: Communication between h1 and h3 via the upper path (S3)

5	19.845205728	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.2
6	20.064514940	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.2
7	21.091256056	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.2
8	22.917892454	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.2
9	23.936498671	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.2
10	24.959805979	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.2
11	25.991624540	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.2
12	25.992709156	00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 10.0.0.4 is at 00:00:00:00:00:04
13	25.993739998	10.0.0.2	10.0.0.4	ICMP	98 Echo (ping) request id=0x0682, seq=1/256, ttl=64 (reply in 14)
14	25.995187563	10.0.0.4	10.0.0.2	ICMP	98 Echo (ping) reply id=0x0682, seq=1/256, ttl=64 (request in 13)
15	31.232405450	00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 Who has 10.0.0.2? Tell 10.0.0.4
16	31.233469894	00:00:00_00:00:02	00:00:00_00:00:04	ARP	42 10.0.0.2 is at 00:00:00:00:00:02
17	32.202639610	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.4
18	33.216380950	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.4
19	34.240378202	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.4
20	35.272094306	10.0.0.4	10.0.0.2	ICMP	98 Echo (ping) request id=0x1a8f, seq=1/256, ttl=64 (reply in 21)
21	35.272554579	10.0.0.2	10.0.0.4	ICMP	98 Echo (ping) reply id=0x1a8f, seq=1/256, ttl=64 (request in 20)
22	35.278429004	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.4
23	36.289502514	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.4
24	37.313437237	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.4

Figure 4: Wireshark capture: Communication between h2 and h4 via the lower path (S4)

2 Service slicing

In this section, we will discuss how it was possible to perform *slicing* of the network in order to differentiate the treatment of different flows. In this particular case, exploiting the same logical view of the network used in the previous section, we ensure that the lower *slice* is intended for normal traffic and the connection of all hosts to each other, while the upper *slice* is intended for video traffic. To perform this separation, filters are implemented on switches s1 and s2. Switch s3 checks that the flow is actually of *streaming video* type (UDP port 9999), while switch s4 handles all other traffic through bidirectional forwarding. No modifications were made to the topology through Mininet, so the changes to the Ryu controller class are presented below.

```

1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     ofproto = datapath.ofproto
5     parser = datapath.ofproto_parser
6     dpid = datapath.id
7     self.logger.info(f"[FEATURES HANDLER] dpid={dpid}")
8     if dpid == 1: # If switch s1, if the packet from the host is UDP on port 9999, send it
        to the upper path.
9         port_up, port_dw = 3, 4
10        port_h1, port_h2 = 1, 2
11        for host_port in [port_h1, port_h2]:
12            match = parser.OFPMatch(
13                in_port=host_port,
14                eth_type=0x0800,
15                ip_proto=17,
16                udp_dst=UDP_PORT_STREAMING
17            )
18            actions = [parser.OFPActionOutput(port_up)]
19            self.add_flow(datapath, priority=100, match=match, actions=actions)
20    elif dpid == 2: # For switch s2, if the packet from the host is UDP on port 9999, send
        it to the upper path.
21        port_up, port_dw = 1, 2
22        port_h3, port_h4 = 3, 4
23        for host_port in [port_h3, port_h4]:
24            match = parser.OFPMatch(
25                in_port=host_port,
26                eth_type=0x0800,
27                ip_proto=17,
28                udp_dst=UDP_PORT_STREAMING
29            )
30            actions = [parser.OFPActionOutput(port_up)]
31            self.add_flow(datapath, priority=100, match=match, actions=actions)
32    elif dpid == 3: # For switch s3, if the packet is UDP on port 9999, let it pass.
33        match = parser.OFPMatch(
34            in_port=1, eth_type=0x0800, ip_proto=17, udp_dst=UDP_PORT_STREAMING
35        )
36        actions = [parser.OFPActionOutput(2)]
37        self.add_flow(datapath, priority=100, match=match, actions=actions)
38        match = parser.OFPMatch(
39            in_port=2, eth_type=0x0800, ip_proto=17, udp_dst=UDP_PORT_STREAMING
40        )
41        actions = [parser.OFPActionOutput(1)]
42        self.add_flow(datapath, priority=100, match=match, actions=actions)
43    elif dpid == 4: # For switch s4, forward all packets bidirectionally.
44        match = parser.OFPMatch(in_port=1)
45        actions = [parser.OFPActionOutput(2)]
46        self.add_flow(datapath, priority=1, match=match, actions=actions)
47        match = parser.OFPMatch(in_port=2)
48        actions = [parser.OFPActionOutput(1)]
49        self.add_flow(datapath, priority=1, match=match, actions=actions)
50    # default rule -> sends packet to controller
51    match = parser.OFPMatch()
52    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
53                                     ofproto.OFPCML_NO_BUFFER)]
54    self.add_flow(datapath, priority=0, match=match, actions=actions)

```

The implemented strategy combines two complementary approaches: the proactive rules installed in the *switch features handler* and reactive management through the *packet in handler*. Proactive rules ensure that video streaming traffic (UDP port 9999) is always routed on the dedicated upper path without involving the controller for each packet. Conversely, reactive management intervenes when no matching rule exists: the packet is forwarded to the controller, which extracts information from the packet header, updates its MAC tables, and decides the optimal path. This hybrid approach reduces latency for critical traffic (video streaming) through proactive rules, while reactive management provides flexibility and adaptability for best effort traffic, thus avoiding overloading the controller with already-classified packets.

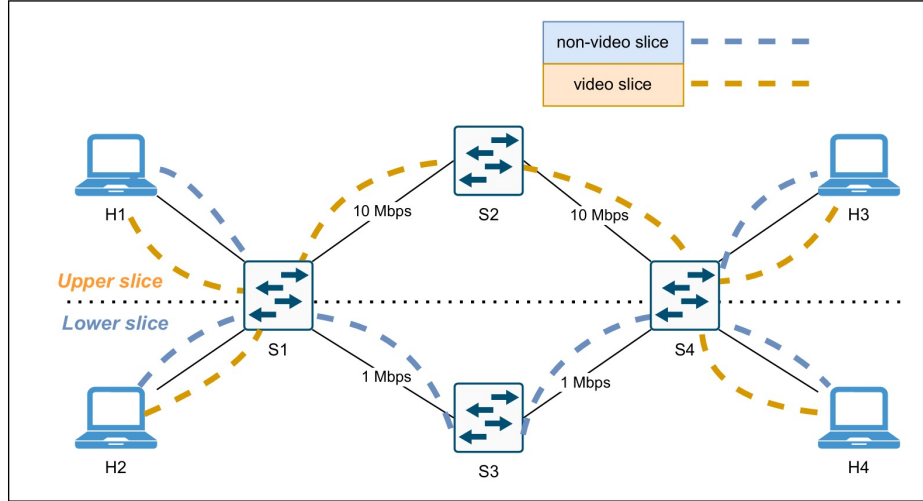


Figure 5: Service slicing logical view.

Service-based forwarding is evident in the instructions concerning packet matching on edge switches, where the rule is defined such that if the packet uses the IP/UDP protocol on port 9999 (video streaming), it is always sent on the upper path, regardless of the MAC addresses of the communicating devices. Switches s3 and s4, on the other hand, are only responsible for bidirectional forwarding in case of a match (IP/UDP on port 9999 for switch s3 and all other packets for switch s4).

```

1 @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
2 def _packet_in_handler(self, ev):
3     msg = ev.msg
4     datapath = msg.datapath
5     ofproto = datapath.ofproto
6     parser = datapath.ofproto_parser
7     dpid = datapath.id
8     in_port = msg.match['in_port'] # port from which the packet was received
9
10    pkt = packet.Packet(msg.data)
11    eth = pkt.get_protocol(ethernet.ethernet)
12    if eth is None:
13        return
14    dst = eth.dst
15    src = eth.src
16    self.mac_to_port.setdefault(dpid, {})
17
18    self.mac_to_port[dpid][src] = in_port # update MAC table with the source MAC and the
        port it came from
19
20    # known destination mac
21    if dst in self.mac_to_port[dpid]:
22        out_port = self.mac_to_port[dpid][dst] # get the port to which the destination MAC
            is associated
23        actions = [parser.OFPActionOutput(out_port)] # create an action to forward the
            packet to the correct port
24
25    # unknown destination mac -> flood only on the down path
26    else:

```

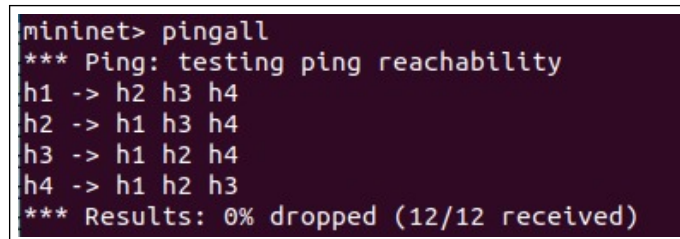
```

27     host_ports = {
28         1: {1,2},
29         2: {3,4},
30         3: set(),
31         4: set()
32     }
33
34     interswitch_links = {
35         1: {4}, # [dpid] -> down path
36         2: {2},
37     }
38
39     actions = []
40     for p in host_ports.get(dpid, set()):
41         if p != in_port:
42             actions.append(parser.OFPACTIONOutput(p))
43
44     for p in interswitch_links.get(dpid, set()):
45         if p != in_port:
46             actions.append(parser.OFPACTIONOutput(p))
47
48     if len(actions) == 1: # if there is only one action, it means we have a known
49         # destination and we can install a flow rule for future packets
50         match = parser.OFPMATCH(in_port=in_port, eth_src=src, eth_dst=dst)
51         self.add_flow(datapath, priority=1, match=match, actions=actions)
52
53     out = parser.OFPPACKETOut(
54         datapath=datapath,
55         buffer_id=msg.buffer_id,
56         in_port=in_port,
57         actions=actions,
58         data=msg.data
59     )
60     datapath.send_msg(out)

```

This function is called every time a switch receives a packet, finds no match, and forwards the packet to the controller. The controller extracts information from the packet and maintains a MAC table for each switch, where the association (MAC, port) is found. If the destination is already known, it is used for direct forwarding. If the destination is unknown, the packet is flooded only on the down route. Then the flow rule is installed on the switch to handle future packets with the same source and destination MAC addresses. Testing of this implementation was performed using Mininet's *pingall* tool, *Wireshark*, and *iPerf* for traffic simulation.

Firstly, the *pingall* tool was used to verify that all hosts can communicate with each other, regardless of the service slicing policies. This is crucial to ensure that the implemented rules do not inadvertently block communication between hosts. The results of the *pingall* test are shown in Figure 6, where all hosts successfully ping each other, confirming that connectivity is maintained across the network.



```

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)

```

Figure 6: pingall results: all hosts can communicate.

For the second test we simulated video streaming traffic using *iPerf* by sending UDP packets on port 9999 from host h4 to host h1. We created a web server on host h1 and a client on host h4. The commands used for this test is shown below:

Command Line

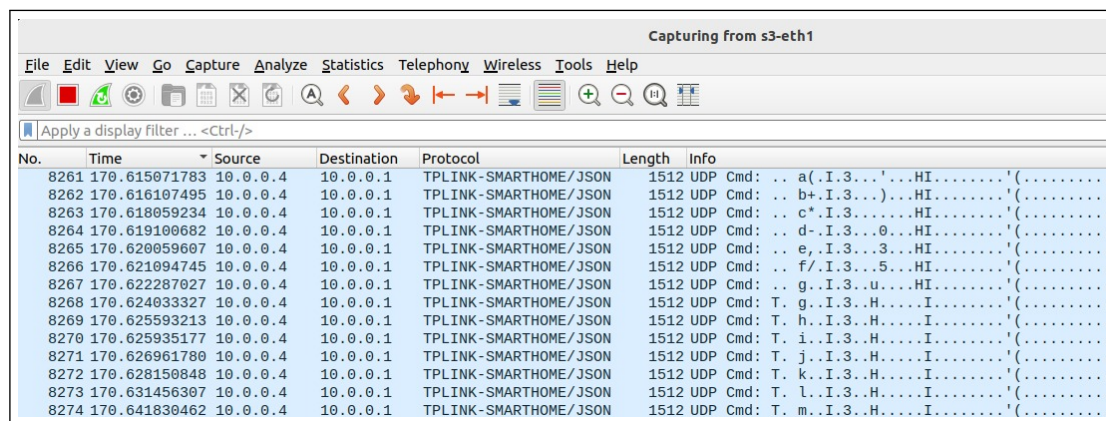
```
mininet> h1 iperf -s -u -p 9999
mininet> h4 iperf -c h1 -u -p 9999 -b 10M -t 10
```

Then we repeated the test for best effort traffic by sending TCP packets on port 5001 from host h4 to host h1, using the following commands:

Command Line

```
mininet> h1 iperf -s -p 5001
mininet> h4 iperf -c h1 -p 5001 -t 10
```

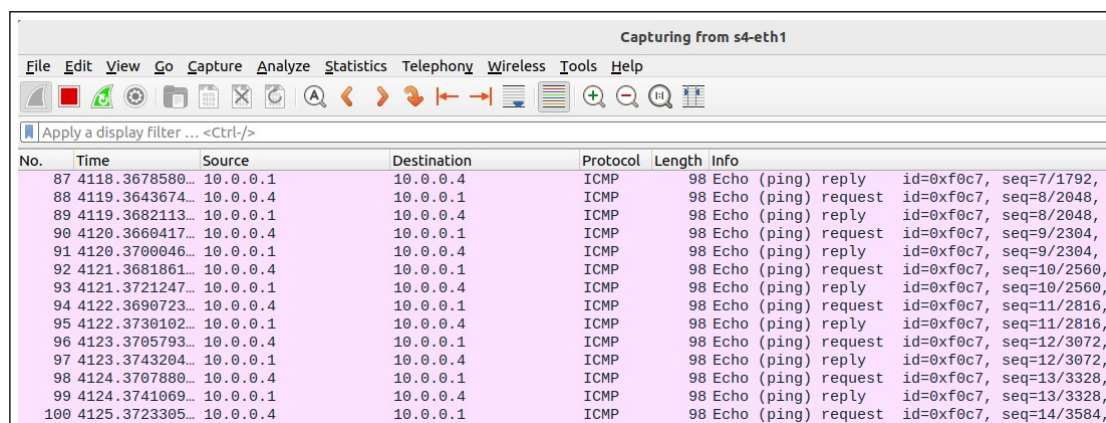
Then Wireshark was used to capture traffic on the interfaces of switches s3 and s4 to verify that the service slicing policies were correctly applied. On switch s3, only video streaming traffic (UDP port 9999) should be visible, while on switch s4, all other traffic should be visible. The Wireshark captures confirm this behavior: on switch s3, only UDP packets on port 9999 are seen (Figure 3), while on switch s4, all other traffic is visible (Figure 4).



The screenshot shows the Wireshark interface capturing traffic on s3-eth1. The packet list table contains 14 entries, all of which are TPLINK-SMARTHOME/JSON packets with a length of 1512 bytes, captured on UDP port 9999. The source and destination IP addresses are 10.0.0.4 and 10.0.0.1 respectively for all packets.

No.	Time	Source	Destination	Protocol	Length	Info
8261	170.615071783	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: .. a(.I.3...'.HI.....' (.....
8262	170.616107495	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: .. b+.I.3...'.HI.....' (.....
8263	170.618059234	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: .. c*.I.3...'.HI.....' (.....
8264	170.619100682	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: .. d-.I.3...'.HI.....' (.....
8265	170.620059607	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: .. e,.I.3...'.HI.....' (.....
8266	170.621094745	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: .. f/.I.3...'.HI.....' (.....
8267	170.622287027	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: .. g.I.3...'.HI.....' (.....
8268	170.624033327	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: T. g.I.3..H....'.HI.....' (.....
8269	170.625593213	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: T. h..I.3..H....'.HI.....' (.....
8270	170.625935177	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: T. i..I.3..H....'.HI.....' (.....
8271	170.626961780	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: T. j..I.3..H....'.HI.....' (.....
8272	170.628150848	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: T. k..I.3..H....'.HI.....' (.....
8273	170.631456307	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: T. l..I.3..H....'.HI.....' (.....
8274	170.641830462	10.0.0.4	10.0.0.1	TPLINK-SMARTHOME/JSON	1512	UDP Cmd: T. m..I.3..H....'.HI.....' (.....

Figure 7: Wireshark capture on S3 interface: all the video streaming traffic (UDP port 9999) is seen on the upper path.



The screenshot shows the Wireshark interface capturing traffic on s4-eth1. The packet list table contains 20 entries, all of which are ICMP Echo (ping) requests and replies, captured on port 5001. The source and destination IP addresses are 10.0.0.4 and 10.0.0.1 respectively for all packets.

No.	Time	Source	Destination	Protocol	Length	Info
87	4118.3678580...	10.0.0.1	10.0.0.4	ICMP	98	Echo (ping) reply id=0xf0c7, seq=7/1792,
88	4119.3643674...	10.0.0.4	10.0.0.1	ICMP	98	Echo (ping) request id=0xf0c7, seq=8/2048,
89	4119.3682113...	10.0.0.1	10.0.0.4	ICMP	98	Echo (ping) reply id=0xf0c7, seq=8/2048,
90	4120.3660417...	10.0.0.4	10.0.0.1	ICMP	98	Echo (ping) request id=0xf0c7, seq=9/2304,
91	4120.3700046...	10.0.0.1	10.0.0.4	ICMP	98	Echo (ping) reply id=0xf0c7, seq=9/2304,
92	4121.3681861...	10.0.0.4	10.0.0.1	ICMP	98	Echo (ping) request id=0xf0c7, seq=10/2560,
93	4121.3721247...	10.0.0.1	10.0.0.4	ICMP	98	Echo (ping) reply id=0xf0c7, seq=10/2560,
94	4122.3690723...	10.0.0.4	10.0.0.1	ICMP	98	Echo (ping) request id=0xf0c7, seq=11/2816,
95	4122.3730102...	10.0.0.1	10.0.0.4	ICMP	98	Echo (ping) reply id=0xf0c7, seq=11/2816,
96	4123.3705793...	10.0.0.4	10.0.0.1	ICMP	98	Echo (ping) request id=0xf0c7, seq=12/3072,
97	4123.3743204...	10.0.0.1	10.0.0.4	ICMP	98	Echo (ping) reply id=0xf0c7, seq=12/3072,
98	4124.3707880...	10.0.0.4	10.0.0.1	ICMP	98	Echo (ping) request id=0xf0c7, seq=13/3328,
99	4124.3741069...	10.0.0.1	10.0.0.4	ICMP	98	Echo (ping) reply id=0xf0c7, seq=13/3328,
100	4125.3723305...	10.0.0.4	10.0.0.1	ICMP	98	Echo (ping) request id=0xf0c7, seq=14/3584,

Figure 8: Wireshark capture on S4 interface: all other traffic is visible.

The second part of the experiment involved simulating both types of traffic simultaneously to verify that the service slicing policies are correctly prioritizing the video streaming traffic. We ran the *iPerf* tests for both video streaming and best effort traffic at the same time, while capturing traffic on h1, where services are hosted.

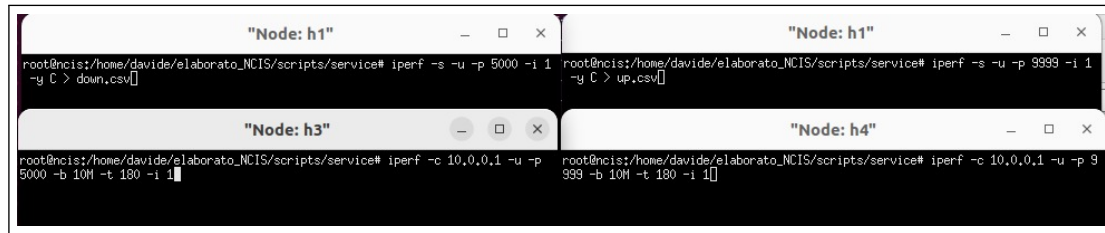


Figure 9: Simultaneous traffic test: video streaming traffic (UDP port 9999) and best effort traffic (UDP port 5001) are generated at the same time.

The *iPerf* results confirm the effectiveness of the implemented strategy: video streaming traffic (UDP port 9999) achieves significantly higher throughput compared to best effort traffic, as shown in Figure 10. This demonstrates that the service slicing policies are correctly differentiating the treatment of flows.

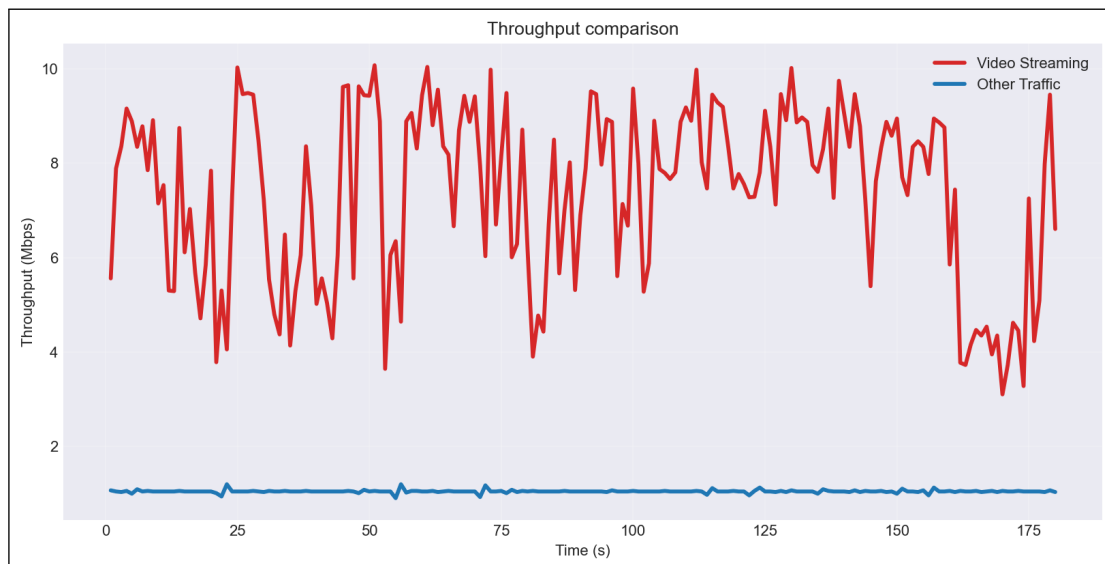


Figure 10: Throughput comparison: video streaming vs best effort traffic.